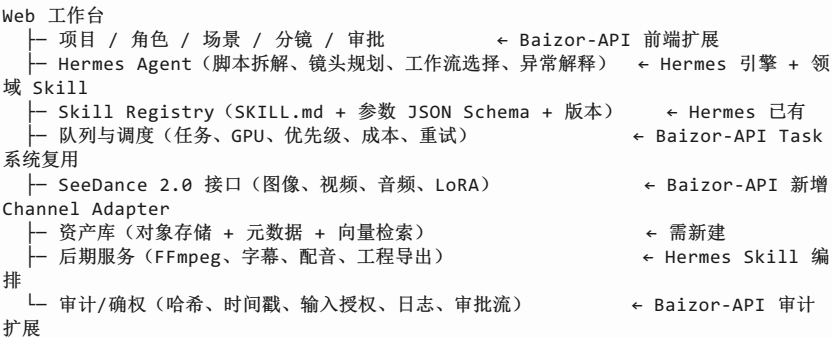


MagicalBrush 技术实施方案

MagicalBrush 技术实施方案 — 基于 Baizor-API + Hermes-Agent

日期：2026-07-08 来源：安妮天算-MagicalBrush 商业计划书 定位：面向影视/动漫/广告的企业级 AIGC 全栈生产平台

一、总体架构



系统分工

系统	角色	已有能力
Baizor-API (Go)	网关 + 调度 + 计费 + 前端 + 数据层	40+ AI 供应商适配、BillingSession 计费、Task 异步队列、RBAC/Team、React 管理台
Hermes-Agent (Python)	AI 大脑 + 工具执行 + 工作流编排	29+ LLM 供应商、109+ 工具、Skill 注册/生命周期、子 Agent 委派、看板协作、MCP 客户端/服务端

两者现有集成点： - Hermes 通过 Baizor-API 的 /v1/chat/completions 调用 LLM (复用路由、计费、重试) - Baizor-API 有 HermesExecutionTask 模型管理 Agent 长任务 - 前端已有 /hermes-playground/ 页面

二、逐模块评估与实施方案

模块 1：Web 工作台（项目 / 角色 / 场景 / 分镜 / 审批）

复用度：70% · 新建量：中等 · ☐ 可行

已有

组件	来源	说明
前端框架	Baizor-API	React 19 + TypeScript + TanStack Router + Tailwind CSS
后端框架	Baizor-API	Go Gin + GORM + Controller/Service/Model 分层
用户/团队	Baizor-API	User + Team + TeamMember (owner/admin/member/viewer)
RBAC	Baizor-API	Casbin 资源/动作权限, 角色授予矩阵, 权限目录 API
Workspace 参考	Baizor-API	/hermes-playground/, /skill-workspace/, /team-workspace/

需新建

后端领域模型（遵循 Baizor-API 的 GORM 模式）：

```
// model/studio_project.go
type StudioProject struct {
    Id          int    `json:"id" gorm:"primaryKey"`
    Name        string `json:"name" gorm:"index"`
    TeamId      int    `json:"team_id" gorm:"index"`
    Status      int    `json:"status" // draft=1, in_progress=2,`
    Brief       string `json:"brief" gorm:"type:text"`
    StyleDNA    string `json:"style_dna" gorm:"type:text" // JSON: 视觉风`
    CreatedAt   int64  `json:"created_at" gorm:"autoCreateTime:milli"`
    UpdatedAt   int64  `json:"updated_at" gorm:"autoUpdateTime:milli"`
}

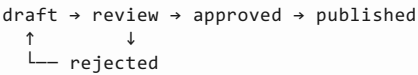
// model/studio_character.go
type StudioCharacter struct {
    Id          int    `json:"id" gorm:"primaryKey"`
    ProjectId   int    `json:"project_id" gorm:"index"`
    Name        string `json:"name"`
    Description string `json:"description" gorm:"type:text"`
    RefImages  string `json:"ref_images" gorm:"type:text" // JSON: 参`
    FaceEmbed  string `json:"face_embed" gorm:"type:text" // IP-Adapt`
    StyleLora   string `json:"style_lora" // 绑定的 Lo`
}

// model/studio_scene.go - 场景
// model/studio_shot.go - 分镜（单个镜头）
// model/studio_approval.go - 审批记录
```

前端路由（新增 /studio/ 路由组）：

```
web/default/src/routes/studio/
├─ index.tsx           - 项目列表
├─ $projectId/
│   ├─ index.tsx       - 项目概览
│   ├─ characters/     - 角色管理
│   ├─ scenes/         - 场景管理
│   ├─ storyboard/     - 分镜编辑器 (Canvas)
│   ├─ timeline/       - 时间线视图
│   ├─ assets/         - 项目资产库
│   ├─ render/         - 渲染任务队列
│   └─ approval/      - 审批面板
```

审批状态机：



基于 Team 角色映射：member 提交 → admin 审批 → owner 发布。

模块 2：Hermes Agent（脚本拆解、镜头规划、 workflow 选择、异常解释）

复用度：95% · 新建量：低 · □ 高度可行

已有（全部直接复用）

能力	来源	说明
Agent 引擎	run_agent.py AI-Agent	90 轮迭代，自动工具调用循环
29+ LLM 供应商	plugins/model-providers/	OpenAI SDK 统一协议
109+ 工具	tools/	终端、文件、浏览器、视觉、图像生成等
子 Agent 委派	tools/delegate_tool.py	单任务 + 批量并行
记忆系统	tools/memory_tool.py	MEMORY.md + USER.md + 8 个插件后端
看板协作	tools/kanban_tools.py	SQLite 看板，多 Agent 任务分配
Skill 系统	tools/skills_tool.py	列表/查看/管理 + Curator 后台维护
MCP 客户端	tools/mcp_tool.py	stdio/HTTP/SSE 传输
Cron	cron/jobs.py	持久化定时任务

需新建：领域 Skill 文件

```
hermes-agent/skills/magicalbrush/
├─ script-analyzer.md      剧本 → 结构化场景/镜头/角色列表
├─ shot-planner.md         场景 → 镜头规划（景别/运镜/时长/构图）
├─ character-designer.md   角色描述 → 参考图 prompt + IP-Adapter 配置
├─ workflow-selector.md    需求 → 自动选择生成管线（图/视频/音频）
├─ quality-checker.md     生成结果 → 质量评分 + 异常诊断 + 修复建议
├─ consistency-guard.md   多镜头角色一致性校验 + 修复指令
├─ post-production.md     素材 → FFmpeg 后期编排（拼接/字幕/配音/调色）
└─ copyright-register.md  成片 → 哈希计算 + 链上存证 + 凭证生成
```

每个 Skill 的结构（遵循 Hermes 现有格式）：

```
---
name: script-analyzer
version: 1.0
description: 将剧本/Brief 拆解为结构化场景和镜头列表
parameters:
  type: object
  properties:
    script_text:
      type: string
      description: 剧本或创意 Brief 文本
    style:
      type: string
      enum: [short_drama, commercial, animation, film]
    output_format:
      type: string
      enum: [json, markdown]
      default: json
    required: [script_text]
  ---
```

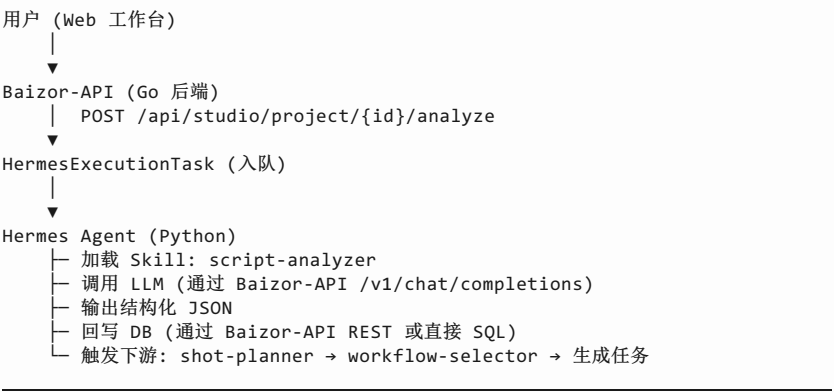
指令

你是一位资深影视编剧和分镜师。分析输入的剧本/Brief，输出结构化的场景和镜头列表。

输出格式（JSON）

```
{
  "title": "...",
  "scenes": [
    {
      "scene_id": 1,
      "location": "室内-咖啡厅-白天",
      "description": "...",
      "shots": [
        {
          "shot_id": "1-1",
          "type": "全景",           // 全景/中景/近景/特写/大特写
          "camera_move": "推",     // 推/拉/摇/移/跟/升/降/固定
          "duration_sec": 3,
          "description": "...",
          "characters": ["角色A"],
          "mood": "温暖",
          "prompt_hint": "..."  // 用于图像/视频生成的 prompt 提示
        }
      ]
    }
  ],
  "characters": [
    { "name": "角色A", "description": "..." }
  ]
}
```

集成架构



模块 3：Skill Registry（SKILL.md + 参数 JSON Schema + 版本）

复用度：90% · 新建量：低 · □ 可行

已有

- tools/skills_tool.py: skill_list, skill_view, skill_manage 三个工具
- agent/curator.py: 后台自动维护 Skill 生命周期状态
- Skill 文件格式: Markdown + YAML frontmatter + 参数定义

需扩展

1. 版本控制: 在 SKILL.md frontmatter 加 version: 1.0 字段
2. 领域分类: 加 category: magicalbrush 字段, 便于分组管理
3. 前端 Skill 市场 (可选): 在 Studio 页面展示可用 Skill 列表、参数说明、版本历史

```
# Skill frontmatter 扩展
---
name: shot-planner
version: 1.2
category: magicalbrush
description: 场景描述 → 分镜规划
parameters: { ... }
---
```

模块 4：队列与调度（任务、GPU、优先级、成本、重试）

复用度：85% · 新建量：低 · □ 高度可行

已有（全部直接复用）

组件	文件	能力
Task 模型	model/task.go	通用异步任务, NOT_START → SUBMITTED → QUEUED → IN_PROGRESS → SUCCESS/FAILURE
任务轮询	service/task_polling.go	后台轮询 + 超时清扫 + TaskPollingAdaptor 接口
分布式锁	model/system_task.go	DB 锁 + 租约 + CAS 更新, 多节点安全
Agent 任务	model/hermes_execution_task.go	Agent 长任务: queued → running → succeeded/failed
计费	service/billing_session.go	pre-consume → settle → refund 全生命周期
通道路由	service/channel_select.go	优先级/权重/分组/failover 重试
成本聚合	model/usedata.go	按用户/模型/通道/Token 聚合用量
表达式计费	pkg/billingexpr/	自定义计费表达式, 支持分层阶梯定价

需扩展

```
// model/task.go - 新增字段
type Task struct {
    // ...existing fields...
    Priority    int    `json:"priority" gorm:"default:0;index"` // 0=norm
    RetryCount int    `json:"retry_count" gorm:"default:0"`
    MaxRetries  int    `json:"max_retries" gorm:"default:3"`
    GpuType     string `json:"gpu_type"` // "A6000
    ProjectId   int    `json:"project_id" gorm:"index"` // 关联 St
}

// constant/channel.go - 新增通道类型
const ChannelTypeSeedance = 50 // SeeDance 2.0

// constant/task.go - 新增任务平台
const TaskPlatformSeedance = "seedance"
const TaskPlatformLoRA     = "lora_training"
```

GPU资源池

通过 Baizor-API 的 **Channel**（通道） 概念抽象 GPU 资源：

```
Channel "seedance-4090-pool"
├─ Type: ChannelTypeSeedance
├─ BaseURL: http://gpu-cluster-1:8080
├─ Priority: 5
├─ Weight: 10
└─ Group: "premium"

Channel "seedance-a6000-pool"
├─ Type: ChannelTypeSeedance
├─ BaseURL: http://gpu-cluster-2:8080
├─ Priority: 10
├─ Weight: 20
└─ Group: "enterprise"
```

任务提交时，channel_select.go 按用户组 + 优先级 + 权重选择 GPU 通道，完全复用现有路由逻辑。

模块 5：SeeDance 2.0 接口（图像、视频、音频、LoRA）

复用度：75% · 新建量：中等 · ☐ 可行

已有参考实现

供应商	适配器路径	可参考点
Kling	relay/channel/task/kling/	最接近的视频生成参考
Sora	relay/channel/task/sora/	OpenAI 风格视频任务
Vidu	relay/channel/task/vidu/	国产视频供应商
Doubao	relay/channel/task/doubao/	火山引擎视频
Hailuo	relay/channel/task/hailuo/	海螺视频
Ali	relay/channel/task/ali/	阿里视频

新建适配器

```
relay/channel/task/seedance/
├─ adaptor.go      - 实现 TaskAdaptor 接口
├─ constants.go   - SeeDance API 常量
├─ dto.go         - 请求/响应 DTO
├─ image.go       - 图像生成
├─ video.go       - 视频生成
├─ audio.go       - 音频/配音
└─ lora.go        - LoRA 训练任务
```

TaskAdaptor 接口实现（参考 Kling 适配器）：

```

type SeedanceAdaptor struct {
    task.TaskAdaptorBase
}

func (a *SeedanceAdaptor) SubmitTask(c *gin.Context, info *relaycommon.Rel
    // 1. 转换请求格式 → SeeDance API 格式
    // 2. 调用 SeeDance API 提交任务
    // 3. 返回任务 ID
}

func (a *SeedanceAdaptor) FetchTask(taskId string, info *relaycommon.Relay
    // 轮询 SeeDance API 获取任务状态
}

```

LoRA 训练

作为特殊任务类型接入：

用户上传训练素材 → 创建 LoRA 训练 Task → 提交到 GPU 通道
 → 轮询训练进度 → 训练完成 → 模型文件入资产库
 → 计费按 GPU 卡时（BillingSession 扩展）

模块 6：资产库（对象存储 + 元数据 + 向量检索）

复用度：30% · 新建量：中高 · □ 可行，需最多新建

已有

- controller/hermes_files.go: 文件上传/下载 API
- Hermes Agent: boto3/S3 引用
- Baizor-API: Embeddings 端点（可生成向量）
- Baizor-API: Redis + 内存缓存

需新建

1. 资产元数据模型

```

// model/studio_asset.go
type StudioAsset struct {
    Id            int    `json:"id" gorm:"primaryKey"`
    ProjectId     int    `json:"project_id" gorm:"index"`
    Name          string `json:"name" gorm:"index"`
    Type          string `json:"type" gorm:"index" // image, video, audio`
    MimeType      string `json:"mime_type" gorm:"index"`
    FileSize      int64  `json:"file_size" gorm:"index"`
    StoragePath   string `json:"storage_path" gorm:"index" // 对象存储路径`
    ThumbnailPath string `json:"thumbnail_path" gorm:"index"`
    ContentHash   string `json:"content_hash" gorm:"index" // SHA-256`
    PHash         string `json:"p_hash" gorm:"index" // 感知哈希 (图像/视频)`
    Tags          string `json:"tags" gorm:"type:text" // JSON 数组`
    Metadata      string `json:"metadata" gorm:"type:text" // JSON: 生成描述`
    CharacterId   *int   `json:"character_id" gorm:"index" // 关联角色`
    SceneId       *int   `json:"scene_id" gorm:"index" // 关联场景`
    ShotId        *int   `json:"shot_id" gorm:"index" // 关联镜头`
    GenerationId  *int   `json:"generation_id" gorm:"index" // 来源生成任务`
    CopyrightId   string `json:"copyright_id" gorm:"index" // 版权存证 ID`
    UserId        int    `json:"user_id" gorm:"index"`
    TeamId        int    `json:"team_id" gorm:"index"`
    Status        int    `json:"status" gorm:"index" // active=1`
    CreatedAt     int64  `json:"created_at" gorm:"autoCreateTime:milli"`
}

```

2. 对象存储抽象层

```
// service/storage/storage.go
type StorageProvider interface {
    Upload(ctx context.Context, path string, reader io.Reader, size int64)
    Download(ctx context.Context, path string) (io.ReadCloser, error)
    Delete(ctx context.Context, path string) error
    GetURL(ctx context.Context, path string, expiry time.Duration) (string)
}

// service/storage/local.go    - 本地文件系统（私有化默认）
// service/storage/minio.go    - MinIO（私有化推荐）
// service/storage/oss.go      - 阿里云 OSS（SaaS 部署）
// service/storage/s3.go       - AWS S3（海外部署）
```

3. 向量检索（一期轻量方案）

```
// model/studio_asset_embedding.go
type StudioAssetEmbedding struct {
    AssetId int    `json:"asset_id" gorm:"primaryKey"`
    Vector  string `json:"vector" gorm:"type:text"` // JSON float 数组
    Model   string `json:"model"` // 使用的 embedding
    UpdatedAt int64 `json:"updated_at"`
}

// service/asset_search.go
// 一期：内存加载 + 余弦相似度暴力搜索（资产量 <10 万够用）
// 二期：接入 pgvector / Milvus
```

通过 Baizor-API 现有 Embeddings 端点生成向量：图像用 CLIP，文本用 text-embedding。

模块 7：后期服务（FFmpeg、字幕、配音、工程导出）

复用度：50% · 新建量：中等 · □ 可行

已有

能力	来源	说明
FFmpeg 执行	Hermes terminal_tool	可运行任意命令行
语音转文字	Baizor-API Whisper 端点	音频 → SRT/文本
文字转语音	Baizor-API TTS 端点	OpenAI TTS + MiniMax TTS
音乐生成	Baizor-API Suno 端点	背景音乐

需新建

FFmpeg Hermes Skill 集合：

```
hermes-agent/skills/magicalbrush/
├─ video-concat.md    - 多片段按序拼接（含转场）
├─ video-overlay.md   - 画面叠加（水印、Logo、字幕烧入）
├─ audio-mix.md       - 音频混合（配音 + 背景音乐 + 音效）
├─ color-grade.md     - 基础调色（LUT 应用）
└─ render-export.md   - 最终渲染输出（分辨率、码率、格式）
```

工程导出（Go 后端服务）：

```
// service/export/edl.go    - EDL (Edit Decision List) 导出（一期）
// service/export/fcpxml.go - Final Cut Pro XML 导出（二期）
// service/export/davinci.go - DaVinci Resolve XML 导出（二期）
```

字幕流水线（Hermes Agent 编排）：

音频/视频 → Whisper 转录 → SRT 生成 → 人工校对（Web 工作台）
→ ASS 样式化 → FFmpeg 烧入 → 输出成片

模块 8：审计/确权（哈希、时间戳、输入授权、日志、审批流）

复用度：60% · 新建量：中等 · □ 可行

已有

能力	来源	说明
操作日志	Baizor-API middleware/	请求审计、访问日志
使用日志	Baizor-API model/usedata.go	按用户/模型/通道聚合
RBAC	Baizor-API Casbin	资源/动作权限
Team 角色	Baizor-API Team 模型	owner/admin/member/viewer

需新建

1. 内容哈希服务

```
// service/copyright/hash.go
func ComputeContentHash(reader io.Reader) (sha256hex string, err error)
func ComputePerceptualHash(imagePath string) (phash string, err error) //
func ComputeVideoFingerprint(videoPath string) (fingerprint string, err er
```

资产入库时自动计算，存入 StudioAsset.ContentHash 和 PHash。

2. 版权存证对接

```
// service/copyright/blockchain.go
type CopyrightService interface {
    Register(ctx context.Context, req *CopyrightRequest) (*CopyrightCert,
        Verify(ctx context.Context, certId string) (*VerifyResult, error)
    Query(ctx context.Context, contentHash string) (*CopyrightCert, error)
}

type CopyrightRequest struct {
    ContentHash string // SHA-256
    PHash       string // 感知哈希
    Title       string // 作品名称
    Author      string // 创作者
    CreatedAt   int64  // 创作时间
    AssetIds    []int  // 关联资产
    InputSources []string // 输入素材来源（审计用）
}

// service/copyright/annie.go - 安妮版权链实现
// service/copyright/antchain.go - 蚂蚁链实现（备选）
```

3. 审批流

```
// model/studio_approval.go
type StudioApproval struct {
    Id int `json:"id" gorm:"primaryKey"`
    ProjectId int `json:"project_id" gorm:"index"`
    TargetType string `json:"target_type" // "project", "shot", "rende
    TargetId int `json:"target_id"`
    Status int `json:"status" // pending=1, approved=2, r
    SubmitterId int `json:"submitter_id"`
    ReviewerId *int `json:"reviewer_id"`
    Comment string `json:"comment" gorm:"type:text"`
    CreatedAt int64 `json:"created_at" gorm:"autoCreateTime:milli"`
    ReviewedAt *int64 `json:"reviewed_at"`
}
```

4. 生成溯源审计

```
// model/studio_generation_log.go - 记录每次 AI 生成的完整上下文
type StudioGenerationLog struct {
    Id            int    `json:"id" gorm:"primaryKey"`
    ProjectId     int    `json:"project_id" gorm:"index"`
    TaskId        string `json:"task_id" gorm:"index"`
    InputAssets   string `json:"input_assets" gorm:"type:text" // JSON:
    InputPrompt   string `json:"input_prompt" gorm:"type:text"`
    ModelUsed     string `json:"model_used"`
    Parameters    string `json:"parameters" gorm:"type:text" // JSON:
    OutputAssets  string `json:"output_assets" gorm:"type:text" // JSON:
    QuotaCost     int    `json:"quota_cost"`
    UserId        int    `json:"user_id" gorm:"index"`
    CreatedAt     int64  `json:"created_at" gorm:"autoCreateTime:milli"`
}
```

三、总体可行性评估

模块	复用度	新建量	可行性	主要工作
Web 工作台	70%	中等	☑	领域模型 + Studio 路由组 +
分镜 Canvas UI				
Hermes Agent	95%	低	☑	5-8 个 SKILL.md 文件
Skill Registry	90%	低	☑	加 version/category 字段
队列与调度	85%	低	☑	Task 加 Priority/GpuType
字段 + 新 Channel Type				
SeeDance 2.0 接口	75%	中等	☑	新 Channel Adapter (参考
Kling)				
资产库	30%	中高	☑	元数据模型 + 对象存储抽象 +
向量检索				
后期服务	50%	中等	☑	FFmpeg Skill 集合 + EDL 导
出				
审计/确权	60%	中等	☑	哈希服务 + 版权链对接 + 审批
流				
整体	~65%	中等	☑ 可行	

结论

方案可行，且复用度显著高于从零搭建。

核心优势： - AI 网关不用重写：Baizor-API 40+ 供应商接入、BillingSession 计费、通道路由、任务队列全部复用 - Agent 引擎不用重写：Hermes 109+ 工具、Skill 注册、子 Agent 编排、记忆系统直接用 - 用户体系不用重写：RBAC、Team、支付、订阅全部复用 - 视频/图像生成管线不用重写：已有 7+ 视频供应商适配器，SeeDance 只需按模式新增

最大新建工作量集中在三块： 1. 领域数据模型（项目/角色/场景/分镜/审批 — ~6 个 Go 文件） 2. 资产库（对象存储抽象 + 元数据 + 向量检索 — ~8 个 Go 文件） 3. 分镜编辑器 UI（Canvas 前端组件 — 前端工作量最大的单项）

四、分阶段实施路线

Phase 1（需要10-12周）：最小闭环 — 剧本到分镜板

目标：用户输入剧本 → Agent 拆解 → SeeDance 生成图像 → 分镜板展示

周	任务	产出
W1-6	Hermes Skill 编写	script-analyzer.md, shot-planner.md, character-designer.md
W7-8	SeeDance Channel Adapter	relay/channel/task/seedance/（图像生成优先）
W9-10	后端领域模型	StudioProject, StudioCharacter, StudioShot, API endpoints
W10-11	前端 Studio 页面	项目创建 + 分镜列表展示（无Canvas，纯列表/网格）
W11-12	端到端联调	全流程跑通，计费验证

Phase 1 验收标准： - 用户在 Web 提交一段剧本文本 - Hermes Agent 自动拆解为场景和镜头列表 - 每个镜头通过 SeeDance 生成对应图像 - 分镜板页面展示所有镜头的图像+描述 - 生成任务在队列中正确流转，计费正确扣减

Phase 2（需要6-8周）：视频生成 + 资产库 + 角色一致性

周	任务	产出
W1-2	SeeDance 视频生成	Adapter 扩展 + 任务轮询
W2-4	资产库后端	StudioAsset 模型 + 对象存储（MinIO/本地）+ CRUD API
W3-5	角色一致性	consistency-guard.md Skill + IP-Adapter 参数管理
W5-7	分镜编辑器 MVP	Konva.js Canvas 编辑器（拖拽排序、属性面板、图像预览）
W7-8	联调测试	视频生成 + 资产入库 + 角色一致性校验

Phase 3（需要6-8周）：后期 + 确权 + 审批 + 私有化

周	任务	产出
W1-2	FFmpeg 后期 Skill	video-concat.md, audio-mix.md, render-export.md
W2-3	字幕流水线	Whisper → SRT → 校对 UI → FFmpeg 烧入
W3-4	版权存证	安妮链 API 对接 + copyright-register.md Skill
W4-6	审批流	StudioApproval 模型 + 审批 UI + 通知
W6-7	工程导出	EDL 导出 + FCP XML 导出
W7-8	私有化部署包	Docker Compose + 本地存储 + 离线配置

五、关键技术决策

决策点	推荐方案	备选	理由
分镜编辑器	Konva.js	Fabric.js	更轻量，React 集成更好
对象存储	MinIO (私有化) + 阿里云 OSS (SaaS)	AWS S3	MinIO S3 兼容，私有化零成本
向量检索	一期内存暴力搜 → 二期 pgvector	Milvus	一期资产量 <10 万无需独立向量库
工程导出	一期 EDL → 二期 FCP XML	DaVinci XML	EDL 最通用，实现简单
版权存证	安妮版权链	蚂蚁链	与甲方（安妮天算）技术栈对齐
LoRA 训练	作为 Task 异步任务	独立训练服务	复用现有 Task 基础设施

六、风险与应对

风险	等级	应对
SeeDance 2.0 API 规范不明确	□ 高	Phase 1 第一周确认 API 文档；若不可用，先用 Kling/DashScope 替代验证流程
角色一致性效果不达标	□ 高	多方案并行（IP-Adapter + LoRA + 后处理修复）；一期保留人工校验兜底
分镜 Canvas 编辑器开发量大	□ 中	Phase 1 用列表/网格视图，Phase 2 再做 Canvas；可评估开源编辑器组件
私有化部署环境碎片化	□ 中	Docker Compose 标准化；预置 NVIDIA Container Toolkit 配置
视频生成质量波动	□ 中	quality-checker.md Skill 自动评分 + 重试；不追求全自动，保留导演干预

七、文件清单汇总

需新建的 Go 文件（Baizor-API 侧）

```

model/
  studio_project.go      - 项目模型
  studio_character.go    - 角色模型
  studio_scene.go        - 场景模型
  studio_shot.go         - 分镜/镜头模型
  studio_asset.go        - 资产元数据模型
  studio_asset_embedding.go - 资产向量模型
  studio_approval.go     - 审批模型
  studio_generation_log.go - 生成溯源日志

controller/
  studio_project.go      - 项目 CRUD API
  studio_character.go    - 角色 API
  studio_storyboard.go   - 分镜 API
  studio_asset.go        - 资产 API
  studio_approval.go     - 审批 API
  studio_render.go       - 渲染任务提交 API

service/
  storage/storage.go     - 对象存储接口
  storage/local.go       - 本地文件存储
  storage/minio.go       - MinIO 存储
  storage/oss.go         - 阿里云 OSS
  copyright/hash.go      - 内容哈希
  copyright/blockchain.go - 版权存证接口
  copyright/annie.go     - 安妮链实现
  asset_search.go        - 资产向量检索
  export/edl.go          - EDL 导出
  export/fcpxml.go       - FCP XML 导出

relay/channel/task/seedance/
  adaptor.go            - SeeDance 任务适配器
  constants.go          - 常量
  dto.go                - DTO
  image.go              - 图像生成
  video.go              - 视频生成

router/
  studio.go             - Studio 路由注册

constant/
  channel.go            - 追加 ChannelTypeSeedance
  task.go               - 追加 TaskPlatformSeedance

```

需新建的 Skill 文件（Hermes-Agent 侧）

```

hermes-agent/skills/magicalbrush/
  script-analyzer.md    - 剧本拆解
  shot-planner.md       - 镜头规划
  character-designer.md  - 角色设计
  workflow-selector.md   - 工作流选择
  quality-checker.md     - 质量检查
  consistency-guard.md   - 角色一致性
  post-production.md     - 后期编排
  copyright-register.md  - 版权存证

```

需新建的前端页面（Baizor-API 前端）

```

web/default/src/routes/studio/
  index.tsx              - 项目列表
  $projectId/
    index.tsx            - 项目概览
    characters/index.tsx - 角色管理
    scenes/index.tsx     - 场景管理
    storyboard/index.tsx - 分镜编辑器
    timeline/index.tsx   - 时间线
    assets/index.tsx     - 资产库
    render/index.tsx     - 渲染队列
    approval/index.tsx   - 审批面板

```

八、验证方式

阶段	验证方法
Skill 单元	Hermes CLI 运行每个 Skill，验证输入/输出格式
Channel 单元	Baizor-API 管理后台添加 SeeDance channel，API Playground 测试
计费验证	Usage Log 检查 SeeDance 调用的 Quota 扣减
端到端	Web 提交剧本 → Agent 拆解 → 生成 → 资产入库 → 审批 → 导出
私有化	Docker Compose 一键部署，断网环境跑通完整流程